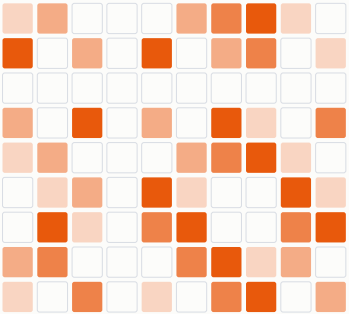
TUTORIAL · UNDERGRADUATE EDITION, 2026

Sparsity and Precision

The two levers that make a language model cheaper to run: touch fewer numbers, and store each number in fewer bits. Mixture of experts, pruning, number formats and quantization, from the ground up. Every term is defined as it appears; a glossary is at the end.

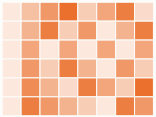


blank: removed four shades: 2-bit values

Two levers on one cost

A model is a huge table of numbers. Cost = how many numbers each token touches × how many bits each has.

as trained



48 numbers, 16 bits each: 768 bits

lever 1: sparsity



skip half of them (blank): 384 bits

lever 2: precision



keep all, 2 bits each (4 shades): 96 bits

cost per token $\propto$ numbers touched × bits per number

INTUITION To write each word the model reads through a giant table of numbers. Make that cheaper in two ways: look at fewer entries, or write each entry with fewer digits.

The accounting

Work. Writing one token costs about 2 FLOPs for every weight it uses: one multiply and one add.

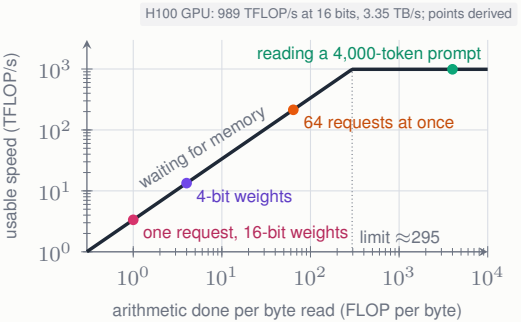
Memory. Weight bytes = weights × bits ÷ 8. A model with 405B weights at 16 bits is 810 GB (derived).

Sparsity uses fewer weights per token. **Precision** stores each one in fewer bits. Both give up a little quality, and both pay off only on hardware that can skip the missing numbers or compute with the short ones.

NEW TERMS **Token**: a word piece, the unit a model reads and writes. **Weight** (parameter): one learned number; B = billion. **FLOP**: one arithmetic operation (an add or a multiply). **Bit**: a binary digit; 8 bits = 1 **byte**; GB = a billion bytes. **Sparsity**: using only some of the numbers. **Precision**: bits per number.

Generation is limited by memory, not arithmetic

To write one token the chip must read every weight it uses, and reading is far slower than computing.



Why the arithmetic sits idle

Each token reads every weight it uses and does 2 FLOPs with it: at 16 bits, 1 FLOP per byte read.

The chip is lopsided. An H100 can compute 989 TFLOP/s but read only 3.35 TB/s.

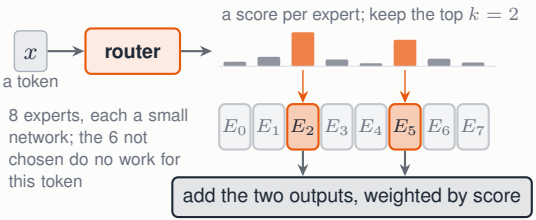
Ways out: read fewer bytes per token, or batch so that one read serves many requests.

INTUITION A chef who can chop 300 carrots a minute, with a helper who fetches one a minute. A sharper knife is useless: fetch less, or feed more tables per trip.

NEW TERMS **Decode**: writing the answer, one token per step. **Prefill**: reading the prompt, all its tokens at once. **GPU**: graphics processing unit, the chip that runs the model. **Memory bandwidth**: bytes it can read per second (T = trillion). **Memory-bound** / **compute-bound**: limited by reading / by arithmetic. **Batch**: requests run together.

Mixture of experts: a few specialists per token

Split one big layer into many small experts; a router sends each token to only a few of them.



How it works

One big layer becomes many small ones. The MLP in each transformer block is split into E small MLPs, the experts.

A router picks. It scores every expert for this token; only the top k run, and their outputs are combined.

Two sizes. Total parameters must fit in memory; active parameters are what one token computes with. DeepSeek-V3: 671B total, 37B active, about 1 in 18.

INTUITION A clinic with many specialists. A triage nurse (the router) sends each patient to the two who fit. Every doctor is on the payroll, but each visit uses only two.

NEW TERMS **MoE**: mixture of experts. **MLP**: multilayer perceptron, the feed-forward part of each transformer block. **Expert**: one of the small MLPs. **Router**: a small learned matrix that scores the experts for a token. **Top-k**: keep the k best scores. **Total** / **active parameters**: all weights / those one token uses.

MoE in practice: keep every expert busy

Left alone, a router overuses a few experts. Training adds a push toward balance.



tokens sent to each of 8 experts in one batch: a collapsed router wastes seven experts and overloads one

INTUITION If the nurse sends everyone to the same two doctors, those two are swamped and the rest sit idle and learn nothing. A good clinic spreads the patients.

why many small experts: choosing 2 of 8 gives 28 combinations; choosing 8 of 32 quarter-size experts gives 10,518,300 for the same work (DeepSeekMoE, 2024)

Balancing the load

The problem. A router that favors a few experts keeps favoring them, and the others never learn.

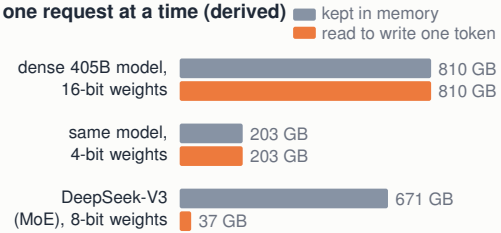
The fix. A push toward equal use during training: an auxiliary loss, or a per-expert bias (DeepSeek-V3).

model (year)	total	active	active %	experts used
Mixtral 8×7B (2023)	47B	13B	28%	2 of 8
DeepSeek-V3 (2024)	671B	37B	5.5%	8 of 256, +1 shared
Kimi K2 (2025)	1.04T	32B	3.1%	8 of 384, +1 shared
gpt-oss-120b (2025)	117B	5.1B	4.4%	4 of 128

NEW TERMS **Load balancing**: keeping the experts about equally used. **Loss**: the number that training tries to make small; an **auxiliary loss** is an extra penalty added to it. **Shared expert**: one that every token uses, next to the chosen ones. **Fine-grained experts**: many small experts instead of a few large ones. T = trillion.

What MoE costs: it trades memory for compute

A token computes with a few experts, but every expert must sit in memory, ready to be picked.



dense: Llama 3.1 405B. With many requests batched together, between them they touch every expert, so the saving in bytes read shrinks.

Reading the bars

Compute follows active. Per token, DeepSeek-V3 does the arithmetic of a 37B model; a dense 405B model does eleven times more.

Memory follows total. All 671 GB must stay loaded, on eight or more GPUs, however few tokens are served.

INTUITION A library with 671 shelves where each question needs only 37 of them. Answering is quick, but you still pay rent for the whole building.

NEW TERMS **Dense model**: one where every weight is used for every token (no experts to skip). **Resident**: loaded in memory, ready to use. **Bytes read per token**: how much memory traffic one decode step needs; it sets the speed when the chip is memory-bound.

Pruning: cut the weights that matter least

Much of a trained network is slack. Remove the smallest weights, retrain, and repeat.



INTUITION Pruning a fruit tree: cut the weakest branches, let the tree recover, cut again. Take too much in one go and it does not recover.

The classic recipe

Train, prune, retrain. Train the full model, set the weights smallest in magnitude to zero, retrain briefly so the rest can compensate, and repeat.

How far? Han et al. (2015) shrank two image networks 9× and 13× with no loss of accuracy this way. In one shot, quality falls off a cliff much earlier.

Lottery tickets (Frankle & Carbin 2019): the surviving 10–20% of weights, reset to their starting values, train alone to full accuracy.

NEW TERMS **Pruning**: setting weights to zero for good, so they need not be stored or used. **Magnitude**: a weight's absolute value; **magnitude pruning** removes the smallest. **Retrain** (fine-tune): continue training briefly after a change. **One-shot**: pruning once, with no retraining. Sparsity is quoted as the share of weights removed.

Structure decides the speedup

A zero saves time only if the hardware can skip it. Scattered zeros rarely can be; regular patterns can.



unstructured
zeros anywhere; one-shot tools reach 50–60% on LLMs

semi-structured 2:4
two zeros in every group of four (the boxes)

structured
whole columns gone; then distill (Minitron: 15B → 8B and 4B)

INTUITION Skipping random words while reading aloud barely saves time: your eyes still pass every line. Skipping whole pages does.

Three patterns, three outcomes

Unstructured: remove any weight (SparseGPT, Wanda). Best quality, but a GPU cannot skip scattered zeros: the file gets smaller, the model barely faster.

2:4: a fixed pattern that NVIDIA GPUs since 2020 can skip: up to 2× on the matrix multiply.

Structured: remove whole neurons or layers. What is left is a smaller ordinary model, fast anywhere, but it loses the most quality and has to be re-taught.

NEW TERMS **Unstructured** / **semi-structured (2:4)** / **structured sparsity**: zeros anywhere / two in every four / whole neurons or layers. **Neuron**: one unit of a layer, one row or column of its weight matrix. **LLM**: large language model. **Distillation**: training a small model to imitate a larger one's outputs.

Sparsity at run time: silent neurons, a smaller cache

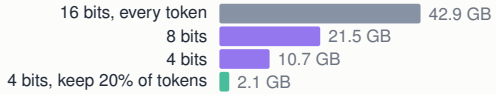
Two more places to skip work: units that output zero, and the stored memory of earlier tokens.

inside one MLP, for one token



3 of 24 units are active; only their weights need to be read

cache for a 128,000-token text, Llama 3 70B (derived)



INTUITION Attention is an open-book exam; the cache is your notes on every earlier token. Notes can be shared, abbreviated, or skimmed.

Skipping work while the model runs

Silent neurons. For any one token many MLP units are zero or close to it. Zero the small ones and skip their weights: TEAL (2024) drops 40–50% and decodes 1.5–1.8× faster.

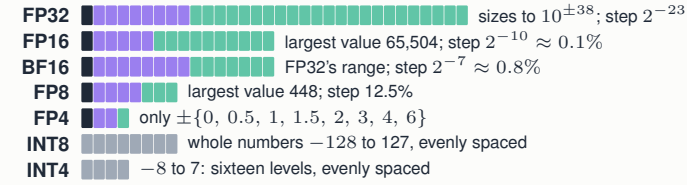
The KV cache keeps a key and a value for every earlier token in every layer. In long texts it, not the weights, is what runs out of memory first.

Shrink it: store less per token (GQA, MLA), use fewer bits, or read only the best-matching tokens (DeepSeek-V3.2 reads the top 2,048).

NEW TERMS **Activation**: a unit's output for one input. **ReLU**: rectified linear unit; it outputs zero for negative inputs. **Attention**: how a token looks up earlier tokens. **KV cache**: key-value cache, the stored keys and values of earlier tokens. **GQA**, **MLA**: grouped-query / multi-head latent attention, two designs that store less per token.

Anatomy of a number

Every format splits a fixed budget of bits: exponent bits buy range, mantissa bits buy fine steps.



INTUITION Scientific notation: in 6.02×10^{23} the exponent (23) says how big, the mantissa (6.02) gives the digits. A format with fewer bits has to choose between a wider range of sizes and more digits.

Reading the rows

Exponent bits buy range. Eight reach $10^{\pm 38}$, four reach 448, two reach 6.

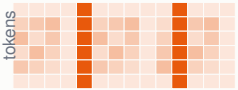
Mantissa bits buy fine steps. The gap between neighboring values is 0.8% for BF16 but 12.5% for FP8.

Below 8 bits a number on its own is too coarse. It works only with a scale shared by a small group (next slide).

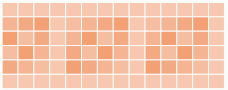
NEW TERMS **Floating point (FP)**: a number stored as a sign, an exponent and a mantissa. **Exponent**: sets the size. **Mantissa**: the significant digits; the **step** is the relative gap between neighboring values. **FP32, FP16, FP8, FP4**: floats of 32, 16, 8, 4 bits. **BF16** (bfloat16): 16 bits with FP32's exponent. **INT8, INT4**: 8- and 4-bit whole numbers.

Scales and outliers

A low-precision number is a small code times a shared scale. One huge value forces a coarse scale on all.



values inside the model: two channels dominate



after a rotation (a reversible remix of channels): spread out

Worked example: 4-bit codes, -8 to 7

Largest value in the group is 0.35, so scale $= 0.35/7 = 0.05$. Then 0.22 is stored as code 4 and read back as 0.20.

Add one outlier of 12: scale $= 12/7 \approx 1.7$, and 0.22 is stored as code 0. It has vanished.

Code $\times$ scale

The trick. Store a small code per number and one scale per group: value $\approx$ scale $\times$ code.

Small groups. MXFP4 and NVFP4 give every block of 32 or 16 numbers its own scale.

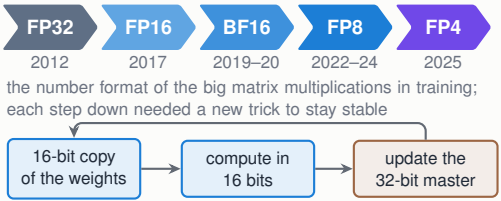
Other fixes: shrink the big channels first (SmoothQuant), or spread them out with a rotation.

INTUITION One ruler for a whole group. If the group holds a hair and a room, marks coarse enough for the room erase the hair. Smaller groups get finer rulers.

NEW TERMS **Quantize**: round a number to the nearest value a short code can hold. **Scale**: the factor that turns a code back into a value. **Outlier channel**: a coordinate of the model's internal vectors with values far larger than the rest. **MXFP4**, **NVFP4**: 4-bit floats with a scale per block of 32 / 16; 4.25 / 4.5 bits per number in all.

Training in fewer bits

Do the heavy arithmetic in short numbers, but keep a full-precision master copy of the weights.



INTUITION A scale that shows whole kilograms cannot register a gain of ten grams a day. Keep one fine scale for the running total; coarse ones are fine for the rest.

What makes it work

A master copy. One update often changes a weight by one part in 10,000 or less. In 16 bits that rounds away and training stalls, so updates are added to a 32-bit copy.

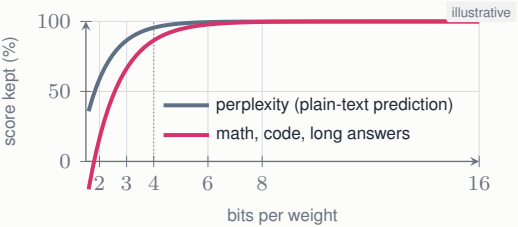
FP8 works when each small block of numbers has its own scale. DeepSeek-V3 (2024) trained this way: loss within 0.25% of a BF16 run.

FP4 also needs stochastic rounding, so that tiny updates survive on average. Shown on a 12B model in 2025.

NEW TERMS **Gradient**: the change each weight should make to lower the loss. **Mixed precision**: short numbers for the arithmetic, long ones where small changes add up. **Master weights**: the 32-bit copy that receives the updates. **Stochastic rounding**: rounding up or down at random, so the average comes out right.

Quantization: fewer bits when the model runs

Round a trained model's weights to 8 or 4 bits. Four bits is where quality starts to fall off.



INTUITION Like saving a photo as a JPEG: a quarter of the size and hard to tell apart. Compress harder and the damage shows, on the hardest tasks first.

Three ways to do it

After training (PTQ). Round the weights layer by layer, checking outputs on sample text (GPTQ, AWQ). Decoding reads fewer bytes, so it speeds up. Four bits gives the best accuracy per bit; three loses (Dettmers & Zettlemoyer 2023).

W8A8: 8-bit activations too; this speeds up the arithmetic itself, on GPUs that support the format.

During training (QAT) when PTQ is not enough. Either way, test on hard tasks, not only perplexity.

NEW TERMS **PTQ** / **QAT**: post-training quantization / quantization-aware training, which trains with the rounding in place; gradients pass through it as if it were not there (the **straight-through estimator**). **W8A8**: 8-bit weights (W) and activations (A). **Perplexity**: a score of how well a model predicts plain text.

Summary: one cost, two levers

Every method here changes how many numbers a token touches, or how many bits each number has.

What to remember

1. Writing a token is limited by reading memory, not by arithmetic. Cut the bytes read per token.

2. MoE trades memory for compute: quality follows total parameters (at a discount), cost follows active ones.

3. A zero saves time only if the hardware can skip it: 2:4 and structured pruning run fast, scattered zeros save space.

4. A short number is a code times a shared scale. Outliers are the difficulty; small blocks are the cure.

5. Train with short arithmetic and a 32-bit master copy. FP8 is proven on a 671B model; FP4 arrived in 2025.

6. To run a model, 4 bits is the knee. Judge the result on hard tasks.

Three models as they ship (derived)

model (kind, weight bits)	in memory	read / token	GPUs
Llama 3.1 405B (dense, 16-bit)	810 GB	810 GB	11
DeepSeek-V3 (MoE, 8-bit)	671 GB	37 GB	9 or more
gpt-oss-120b (MoE, 4-bit experts)	∼65 GB	∼3 GB	1

one request at a time. GPUs: 80 GB cards needed for the weights alone; real deployments need more.

The pattern, and what is still new

Train sparse (MoE) and in FP8, ship the experts at 4 bits, shrink the cache: the savings multiply. gpt-oss-120b holds 117B weights yet decodes at the cost of a 5B model, on one GPU. Training in FP4 is still a research setting in 2026.

Glossary, part 1

Every term and abbreviation from the NEW TERMS strips, in alphabetical order: A to Me.

Activation a unit's output for one input.

Active / total parameters the weights one token uses / all the weights.

Attention the part of a transformer where a token looks up earlier tokens.

Auxiliary loss an extra penalty added to the training loss.

Batch requests processed together.

BF16 bfloat16, a 16-bit float that keeps FP32's exponent.

Bit, byte, GB a binary digit; 8 bits; a billion bytes.

Decode / prefill writing the answer token by token / reading the prompt at once.

Dense model one where every weight is used for every token.

Distillation training a small model to imitate a larger one.

Expert one of the small MLPs in a mixture-of-experts layer.

Exponent, mantissa the part of a float that sets its size / holds its digits.

Fine-grained experts many small experts instead of a few large ones.

FLOP one floating-point operation: an add or a multiply.

Floating point (FP) sign, exponent, mantissa. FP32, FP16, FP8, FP4 use 32 to 4 bits.

GPU graphics processing unit, the chip that runs the model.

GQA, MLA grouped-query / multi-head latent attention: store less per token.

Gradient the change each weight should make to lower the loss.

INT8, INT4 8- and 4-bit whole numbers.

KV cache key-value cache: the stored keys and values of earlier tokens.

LLM large language model.

Load balancing keeping the experts about equally used.

Loss the number that training tries to make small.

Magnitude pruning removing the weights smallest in absolute value.

Master weights the 32-bit copy of the weights that receives the updates.

Memory bandwidth bytes the chip can read from memory per second.

Memory-bound / compute-bound limited by reading memory / by arithmetic.

Glossary, part 2

Mi to W.

Mixed precision short numbers for the arithmetic, long ones where small changes add up.

MLP multilayer perceptron, the feed-forward part of each transformer block.

MoE mixture of experts: many small MLPs, a few used per token.

MXFP4, NVFP4 4-bit floats with one scale per block of 32 / 16 numbers.

Neuron one unit of a layer.

One-shot done once, with no retraining afterwards.

Outlier channel a coordinate of the model's vectors with values far above the rest.

Perplexity a score of how well a model predicts plain text.

Precision how many bits store each number.

Pruning setting weights to zero for good.

PTQ / **QAT** post-training quantization / quantization-aware training.

Quantize round a number to the nearest value a short code can hold.

ReLU rectified linear unit: outputs zero for negative inputs.

Resident loaded in memory, ready to use.

Retrain (fine-tune) continue training briefly after a change.

Rotation a reversible mixing of the coordinates of a vector.

Router the small matrix that scores the experts for each token.

Scale the factor that turns a stored code back into a value.

Shared expert an expert that every token uses.

Sparsity using only some of the numbers.

Stochastic rounding rounding up or down at random, so the average comes out right.

Straight-through estimator in QAT, passing gradients through the rounding as if it were not there.

Structured / **unstructured** / **2:4 sparsity** whole neurons or layers / zeros anywhere / two zeros in every four.

Token a word piece, the unit a model reads and writes.

Top-k keep the k highest scores.

W8A8 8-bit weights (W) and 8-bit activations (A).

Weight (parameter) one learned number in the model. B = billion, T = trillion.

Sources and further reading

The papers and vendor material cited on these slides; † marks vendor pages, model cards and blogs.

Cost model and hardware. NVIDIA H100 datasheet (989 TFLOP/s, 3.35 TB/s)†. Llama 3.1 405B and DeepSeek-V3 published configurations.

Mixture of experts. Fedus et al. 2021, Switch Transformer (auxiliary loss), arXiv 2101.03961. Jiang et al. 2024, Mixtral, 2401.04088. Dai et al. 2024, DeepSeekMoE, 2401.06066. Wang et al. 2024, balancing without an auxiliary loss, 2408.15664. DeepSeek-V3, 2412.19437. Kimi K2, 2507.20534. gpt-oss model card, 2508.10925†.

Pruning. Han et al. 2015, 1506.02626. Frankle & Carbin 2019, lottery tickets, 1803.03635. Mishra et al. 2021, 2:4 sparsity, 2104.08378. Frantar & Alistarh 2023, SparseGPT, 2301.00774. Sun et al. 2023, Wanda, 2306.11695. Muralidharan et al. 2024, Minitron, 2407.14679.

Sparsity at run time. Li et al. 2022, lazy neurons, 2210.06313. Liu et al. 2024, TEAL, 2408.14690. Ainslie et al. 2023, GQA, 2305.13245. DeepSeek-V2 (MLA), 2405.04434. Zhang et al. 2023, H2O, 2306.14048. Liu et al. 2024, KIVI, 2402.02750. DeepSeek-V3.2 (sparse attention), 2512.02556.

Number formats and outliers. Micikevicius et al. 2022, FP8 formats, 2209.05433. OCP Microscaling Formats (MX) specification v1.0, 2023†. NVIDIA, NVFP4 developer blog, 2025†. Dettmers et al. 2022, LLM.int8(), 2208.07339. Xiao et al. 2022, SmoothQuant, 2211.10438. Ashkboos et al. 2024, QuaRot, 2404.00456. Liu et al. 2024, SpinQuant, 2405.16406.

Low-precision training. Micikevicius et al. 2018, mixed precision, 1710.03740. DeepSeek-V3, Sec. 3.3 (FP8 training), 2412.19437. Tseng et al. 2025, MXFP4 training, 2502.20586. NVIDIA 2025, NVFP4 pretraining, 2509.25149.

Quantization for inference. Frantar et al. 2022, GPTQ, 2210.17323. Lin et al. 2023, AWQ, 2306.00978. Dettmers & Zettlemoyer 2023, the case for 4-bit precision, 2212.09720. Liu et al. 2023, LLM-QAT, 2305.17888.

Curves labelled *illustrative* are sketches of a mechanism, not data. Numbers labelled *derived* were computed from published configurations (GB $= 10^9$ bytes). The worked example on slide 11 uses made-up values to show the arithmetic.

The full reference list, with every paper behind the longer treatment, is on the sources slide of the original 33-slide deck, `sparsity_precision_tutorial.tex`.